

LAB 05 – DANH SÁCH SẢN PHẨM VỚI DỮ LIỆU CƠ SỞ DỮ LIỆU

5.1. CẤU HÌNH CƠ SỞ DỮ LIỆU MySQL

5.1.1. GIỚI THIỆU VỀ MySQL

MySQL là hệ quản trị cơ sở dữ liệu SQL nguồn mở phổ biến nhất được phát triển, phân phối và hỗ trợ bởi Oracle.

Cơ sở dữ liệu là một tập hợp dữ liệu có cấu trúc. Nó có thể là bất cứ thứ gì từ một danh sách mua sắm đơn giản đến một thư viện hình ảnh hoặc lượng thông tin khổng lồ trong mạng công ty. Để thêm, truy cập và xử lý dữ liệu được lưu trữ trong cơ sở dữ liệu máy tính, cần có hệ quản trị cơ sở dữ liệu như MySQL.

Cơ sở dữ liệu MySQL có tính chất quan hệ. Cơ sở dữ liệu quan hệ lưu trữ dữ liệu trong các bảng riêng biệt thay vì đặt tất cả dữ liệu vào một kho lớn. Cấu trúc cơ sở dữ liệu được tổ chức thành các tập tin vật lý được tối ưu hóa về tốc độ. MySQL cung cấp một model logic với các đối tượng như databases, tables, views, rows, và columns, cung cấp một môi trường lập trình linh hoạt.

5.1.2. MySQL TABLES

Bảng được sử dụng để sắp xếp dữ liệu theo hàng và cột. Nó được sử dụng để lưu trữ và hiển thị các bản ghi ở định dạng có cấu trúc. Nó giống như các bảng tính trong ứng dụng bảng tính. Các cột chỉ định loại dữ liệu, trong khi các hàng chứa dữ liệu thực tế. Dưới đây là cách có thể tưởng tượng một bảng MySQL (xem Hình 10-1).

id	name	description	image	price
1	TV	Best tv	game.png	1000
2	iPhone	Best iPhone	safe.png	999
3	Chromecast	Best Chromecast	submarine.png	30
4	Glasses	Best Glasses	game.png	100

Hình 5-1. Product table.

5.1.3. GIỚI THIỆU VỀ CƠ SỞ DỮ LIỆU LARAVEL

Hầu hết mọi ứng dụng web hiện đại đều tương tác với cơ sở dữ liệu. Laravel làm cho việc tương tác với cơ sở dữ liệu trở nên đơn giản trên nhiều cơ sở dữ liệu được hỗ trợ. Laravel cung cấp hỗ trợ cho bốn cơ sở dữ liệu:

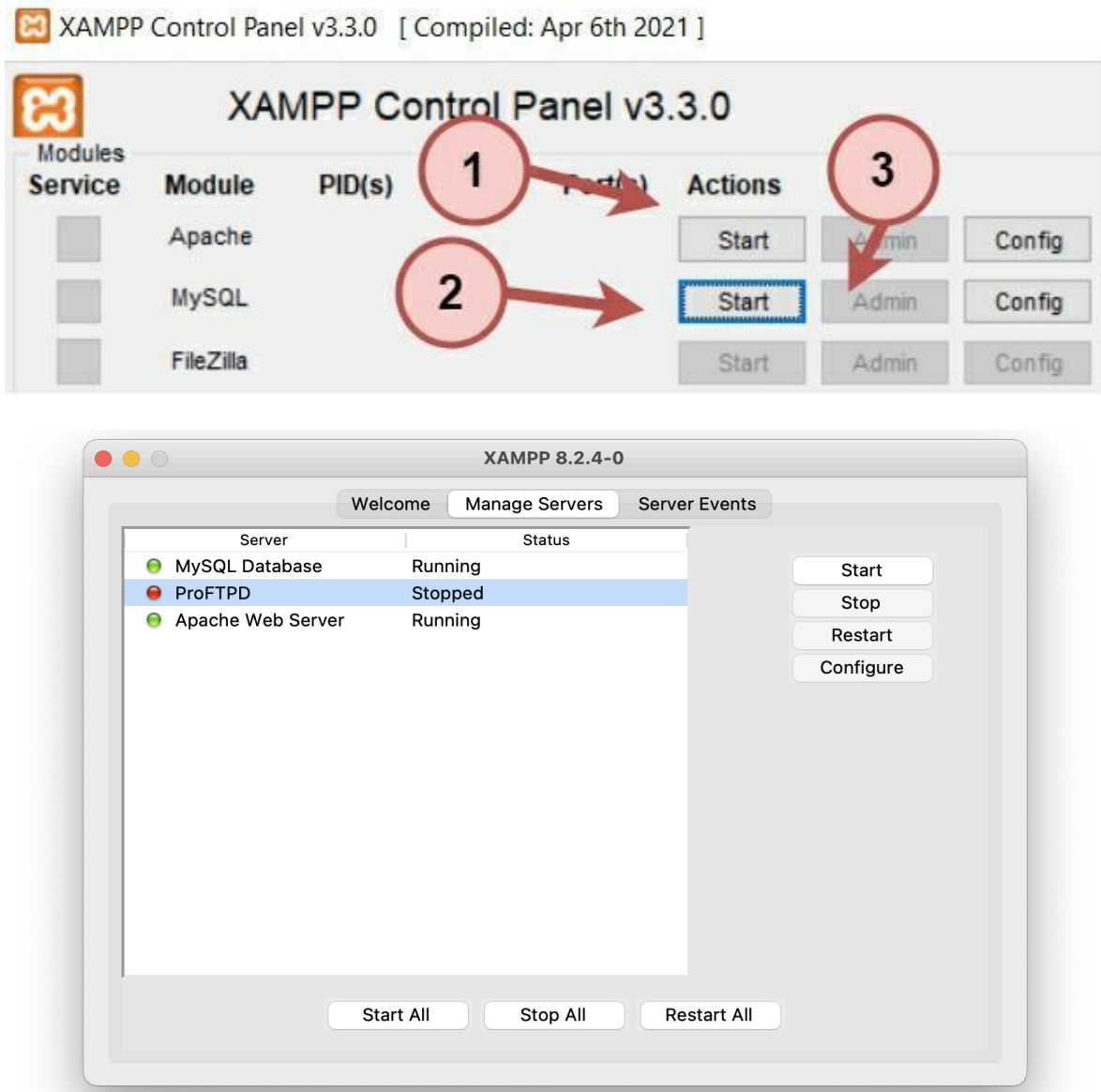
- MySQL 5.7+
- PostgreSQL 9.6+
- SQLite 3.8.8+
- SQL Server 2017+

Trong dự án này, chúng ta sẽ làm việc với MySQL. Chúng ta sẽ sử dụng mô-đun MySQL mà

XAMPP (WAMP) cung cấp.

5.1.4. ĐỊNH CẤU HÌNH CƠ SỞ DỮ LIỆU

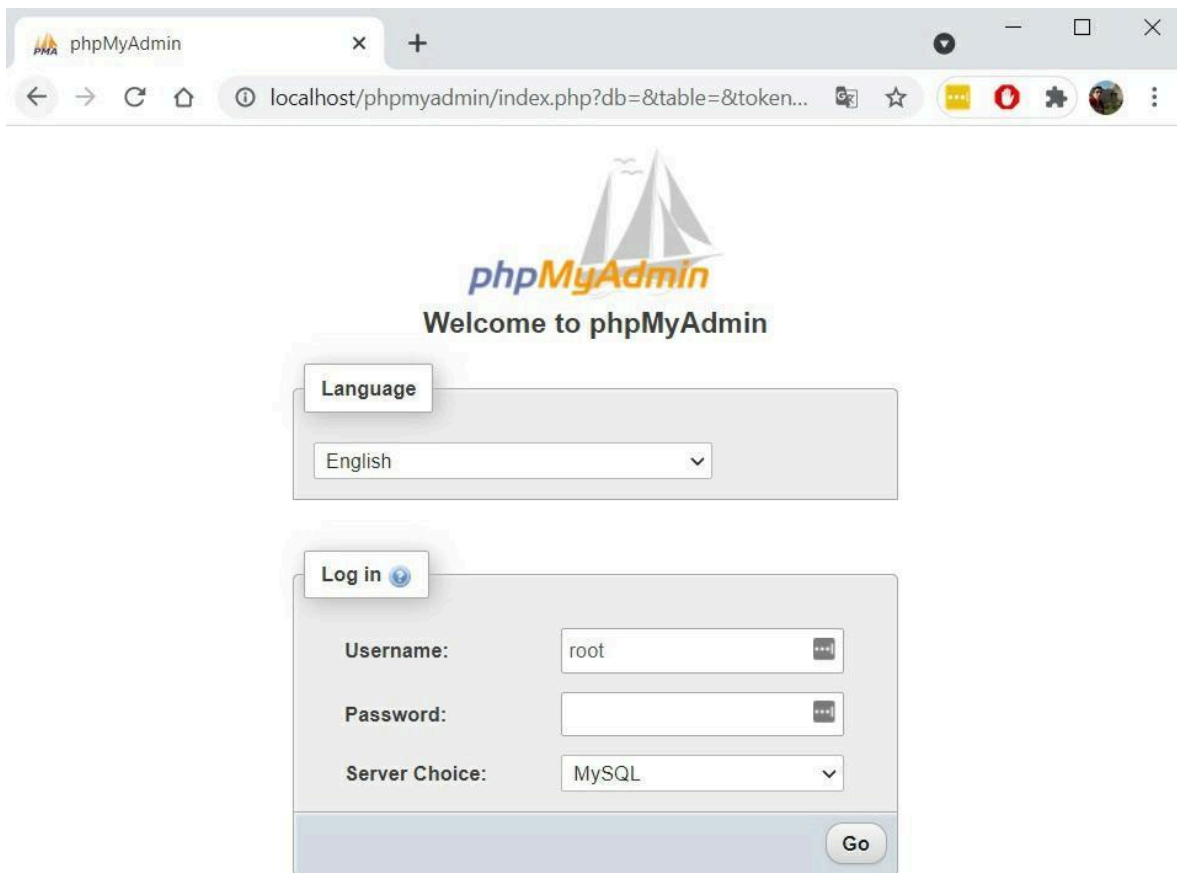
Chạy XAMPP, khởi động mô-đun Apache, khởi động mô-đun MySQL và nhấp vào nút MySQL Admin (của mô-đun MySQL). Nó đưa chúng ta đến ứng dụng phpMyAdmin (xem Hình 5-2).



Hình 5-2. Khởi động mô-đun MySQL trong XAMPP.

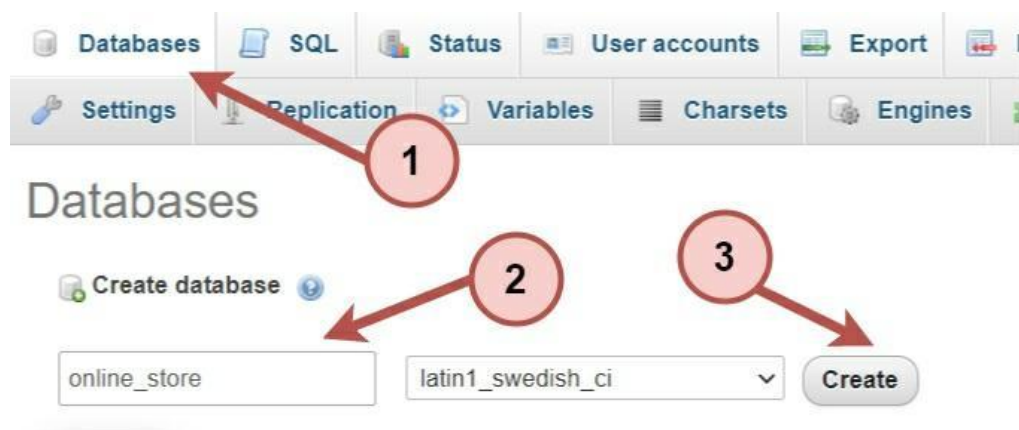
Lưu ý: Nếu đang sử dụng WAMP hoặc một ứng dụng tương tự khác, ứng dụng phpMyAdmin có thể được truy cập thông thường thông qua địa chỉ: <http://localhost/phpmyadmin/>.

Trong ứng dụng phpMyAdmin, nhập tên người dùng và mật khẩu. Giá trị mặc định là “root” (đối với tên người dùng) và mật khẩu trống (xem Hình 5-3).



Hình 5-3. Ứng dụng XAMPP phpMyAdmin.

Sau khi đăng nhập vào phpMyAdmin, hãy nhấp vào tab Databases. Nhập tên cơ sở dữ liệu “online_store” và nhấp vào nút Create (xem Hình 5-4).



Hình 5-4. Tạo cơ sở dữ liệu.

Chúng ta sẽ sử dụng tên người dùng, mật khẩu và cơ sở dữ liệu trong chương tiếp theo.

5.2. PRODUCT MIGRATION

5.2.1. GIỚI THIỆU LARAVEL MIGRATIONS

Bây giờ, chúng ta chỉ cần truy cập ứng dụng phpMyAdmin và tạo các bảng bên trong cơ sở dữ liệu online_store (chẳng hạn như bảng sản phẩm). Nó có thể được thực hiện thông qua việc điền vào biểu mẫu. Đó là cách truyền thống. Tuy nhiên, nó có một vấn đề, nó không cho phép chúng ta kiểm soát phiên bản của các bảng và truy vấn cơ sở dữ liệu. Nếu đã từng phải yêu cầu một đồng đội thêm cột bảng vào lược đồ cơ sở dữ liệu cục bộ của họ theo cách thủ công sau khi lấy các thay đổi của bạn từ source (chẳng hạn như GitHub), thì đã gặp phải vấn đề.

Laravel migrations giống như kiểm soát phiên bản cho cơ sở dữ liệu của ứng dụng. Chúng giải quyết vấn đề trước đó cho phép chúng ta xác định và chia sẻ định nghĩa lược đồ cơ sở dữ liệu của ứng dụng.

Chúng ta sẽ không tạo bảng cơ sở dữ liệu theo cách truyền thống. Thay vào đó, chúng ta sẽ thực hiện điều đó thông qua Laravel migrations.

5.2.2 PRODUCT MIGRATION

Hãy tạo product migration. Trong Terminal, đi tới thư mục dự án và thực hiện như sau:

Terminal

```
php artisan make:migration create_products_table
```

Lệnh trên tạo tập tin products table migration bên trong thư mục database/migrations. Mỗi tên tập tin migration chứa dấu thời gian cho phép Laravel xác định thứ tự migrations. Trong trường hợp của chúng ta, nó đã tạo một tập tin có tên 2024_07_21_094149_create_products_table.php.

Nếu nhìn vào tập tin được tạo, nó có chứa lớp migration. Các lớp migration chứa hai phương thức: up và down. Phương thức up được sử dụng để thêm bảng, cột hoặc chỉ mục mới vào cơ sở dữ liệu. Ngược lại, phương thức down sẽ đảo ngược các thao tác được thực hiện bởi phương thức up.

Bây giờ hãy mở tập tin được tạo trước đó. Xóa tất cả code hiện có và điền vào code sau.

Code

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
}
```

```
*/
public function up(): void
{
    Schema::create('products', function (Blueprint $table) {
        $table->id();
        $table->string('name');
        $table->text('description');
        $table->string('image');
        $table->integer('price');
        $table->timestamps();
    });
}

/**
 * Reverse the migrations.
 */
public function down(): void
{
    Schema::dropIfExists('products');
}
};
```

Hãy phân tích code trước đó theo từng phần.

Code

```
public function up(): void
{
    Schema::create('products', function (Blueprint $table) {
        $table->id();
        $table->string('name');
        $table->text('description');
        $table->string('image');
        $table->integer('price');

        $table->timestamps();
    });
}
```

Phương thức up sẽ tạo một database table mới có tên products. Theo mặc định, Laravel gợi ý tạo tên bảng ở số nhiều. Đó là do cách hoạt động của hệ thống Laravel ORM “Eloquent” (sẽ thảo luận sau). Phương thức up xác định việc tạo bảng sản phẩm với năm cột (id, name, description, image và price). Do đó, phương thức timestamps sẽ thêm hai cột (create_at và update_at). Chúng ta cũng sử dụng năm loại kiểu dữ liệu cột (id, string, text, integer, và timestamps). Có thể tìm thêm thông tin về các loại kiểu dữ liệu cột có sẵn tại đây:

<https://laravel.com/docs/11.x/migrations#available-column-types>.

Code

```
public function down() {
    Schema::dropIfExists('products');
}
```

Phương thức down có tính chất ngược lại với phương thức up. Nó drop bảng products. Chúng ta sẽ xem cách thực hiện các phương thức up và down của các lớp migration sau. Nhưng trước tiên, chúng ta cần cập nhật thông tin đăng nhập cơ sở dữ liệu trong dự án Laravel.

Luôn xác định lược đồ cơ sở dữ liệu bằng cách dùng migrations hoặc các phương pháp tương tự. Hãy nhớ rằng, chúng hoạt động như kiểm soát phiên bản cơ sở dữ liệu. Hầu hết các khung ứng dụng web đều cung cấp các loại tính năng này. Ví dụ: Django cung cấp Django Migrations hoạt động giống như cách tiếp cận trước đó.

5.2.3. SỬA ĐỔI TẬP TIN .ENV

Để thực hiện migrations, chúng ta cần sửa đổi tập tin .env (nằm ở thư mục gốc của dự án). Trong tập tin .env, hãy in đậm những thay đổi sau. Cần đặt DB_DATABASE, DB_USERNAME và DB_PASSWORD. Nếu có tên cơ sở dữ liệu, tên người dùng hoặc mật khẩu khác, hãy thực hiện các thay đổi tương ứng.

Code

```
...
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=online_store
DB_USERNAME=root
DB_PASSWORD=
...
```

5.2.4. THỰC HIỆN MIGRATIONS

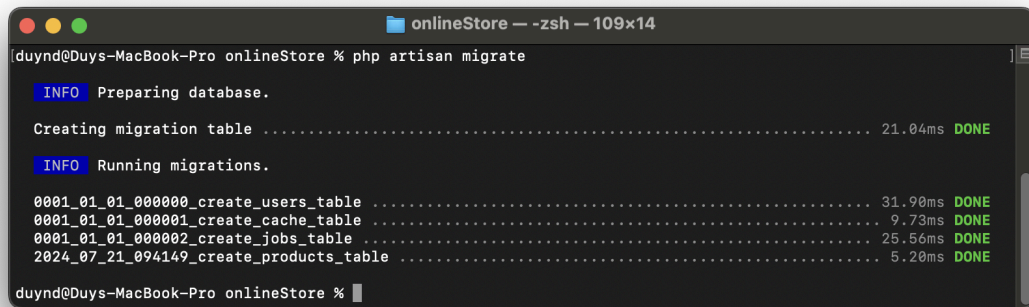
Để chạy migrations, trong Terminal, hãy chuyển đến thư mục dự án và thực hiện như sau:

Terminal

```
php artisan migrate
```

Lệnh trên thực thi các migrations được xác định trong thư mục database/migrations. Năm migrations được thực hiện. Phần cuối cùng là migrations "tạo bảng products". Các migrations khác tương ứng với các migrations mặc định của Laravel. Chúng ta sẽ tận dụng một số trong số

chúng trong các phần sắp tới. Nếu mọi thứ hoạt động như mong đợi, sẽ thấy kết quả như được trình bày trong Hình 5-5.



```
duynd@Duys-MacBook-Pro onlineStore % php artisan migrate

INFO: Preparing database.

Creating migration table ..... 21.04ms DONE

INFO: Running migrations.

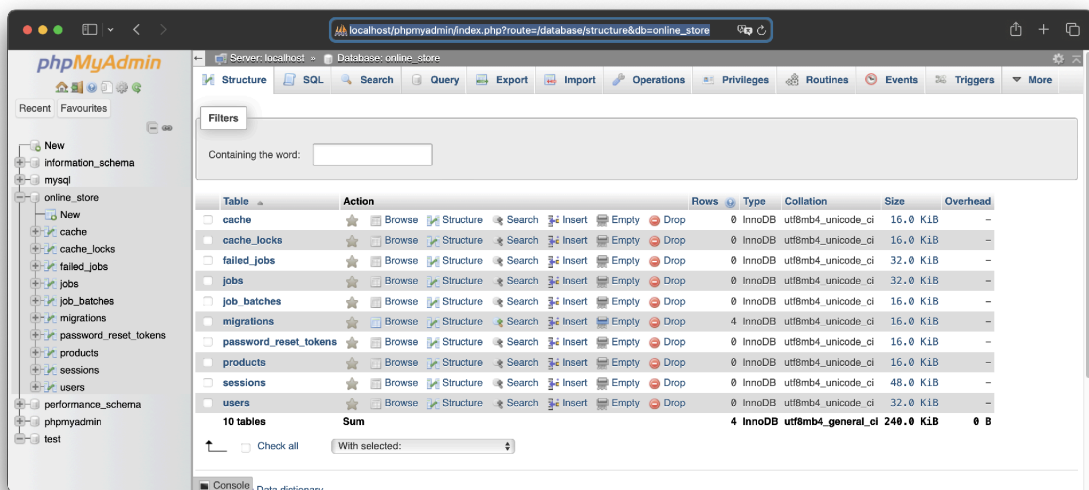
0001_01_01_000000_create_users_table ..... 31.90ms DONE
0001_01_01_000001_create_cache_table ..... 9.73ms DONE
0001_01_01_000002_create_jobs_table ..... 25.56ms DONE
2024_07_21_094149_create_products_table ..... 5.20ms DONE

duynd@Duys-MacBook-Pro onlineStore %
```

Hình 5-5. Thực hiện migrations Laravel.

Xác minh migrations trong phpMyAdmin

Đã đến lúc xác minh rằng việc migrations đã được áp dụng đúng cách trong cơ sở dữ liệu. Đi tới ứng dụng phpMyAdmin và nhấp qua cơ sở dữ liệu online_store (xem Hình 11-2).



Hình 5-6. Truy cập cơ sở dữ liệu từ phpMyAdmin.

Như có thể thấy, bảng sản phẩm đã được tạo, migrations đã hoạt động.

5.2.5. THÊM SẢN PHẨM

Hãy thêm bốn sản phẩm vào cơ sở dữ liệu. Hiện tại, chúng ta sẽ thêm nó theo cách thủ công (thông qua truy vấn SQL). Chúng ta có thể thêm sản phẩm thông qua một biểu mẫu trong phần sau.

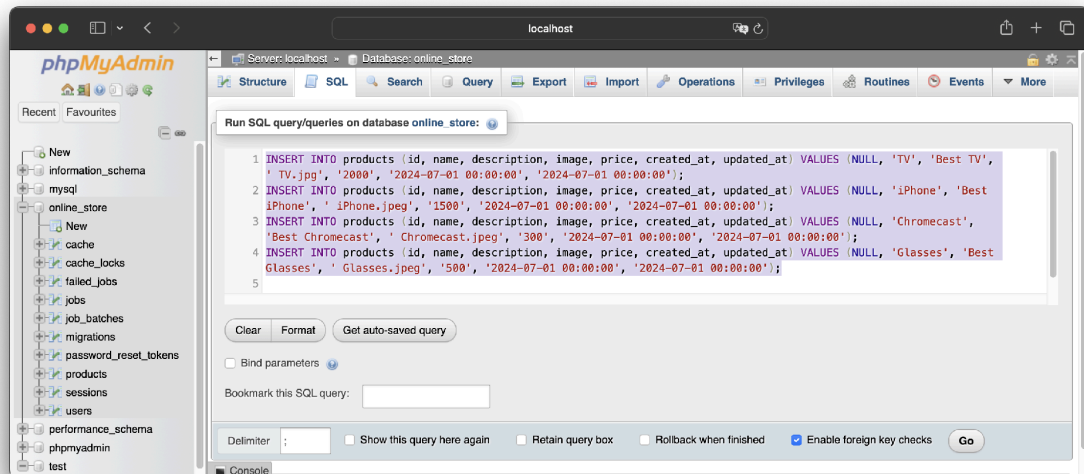
Trong phpMyAdmin, nhấp vào cơ sở dữ liệu online_store, nhấp vào tab SQL, dán các truy vấn SQL sau và nhấp vào go (xem Hình 5-7).

Thêm Database

```

INSERT INTO products (id, name, description, image, price, created_at, updated_at)
VALUES (NULL, 'TV', 'Best TV', ' TV.jpg', '2000', '2024-07-01 00:00:00',
'2024-07-01 00:00:00');
INSERT INTO products (id, name, description, image, price, created_at, updated_at)
VALUES (NULL, 'iPhone', 'Best iPhone', ' iPhone.jpeg', '1500', '2024-07-01
00:00:00', '2024-07-01 00:00:00');
INSERT INTO products (id, name, description, image, price, created_at, updated_at)
VALUES (NULL, 'Chromecast', 'Best Chromecast', ' Chromecast.jpeg', '300',
'2024-07-01 00:00:00', '2024-07-01 00:00:00');
INSERT INTO products (id, name, description, image, price, created_at, updated_at)
VALUES (NULL, 'Glasses', 'Best Glasses', ' Glasses.jpeg', '500', '2024-07-01
00:00:00', '2024-07-01 00:00:00');

```



Hình 5-7. Thêm sản phẩm thông qua phpMyAdmin.

Hãy kiểm tra xem các sản phẩm đã được thêm thành công chưa. Trong phpMyAdmin, nhấp vào cơ sở dữ liệu online_store và nhấp vào bảng sản phẩm. Hy vọng sẽ thấy bốn sản phẩm được thêm vào (xem Hình 5-8).

				id	name	description	image	price	created_at	updated_at
☐	Edit	Copy	Delete	1	TV	Best TV	TV.jpg	2000	2024-07-01 00:00:00	2024-07-01 00:00:00
☐	Edit	Copy	Delete	2	iPhone	Best iPhone	iPhone.jpeg	1500	2024-07-01 00:00:00	2024-07-01 00:00:00
☐	Edit	Copy	Delete	3	Chromecast	Best Chromecast	Chromecast.jpeg	300	2024-07-01 00:00:00	2024-07-01 00:00:00
☐	Edit	Copy	Delete	4	Glasses	Best Glasses	Glasses.jpeg	500	2024-07-01 00:00:00	2024-07-01 00:00:00

Hình 5-8. Sản phẩm được hiển thị trong phpMyAdmin.

5.3. PRODUCT MODEL

5.3.1. GIỚI THIỆU VỀ ELOQUENT

Eloquent là một trình ánh xạ quan hệ đối tượng (ORM - object-relational map) của Laravel giúp

việc tương tác với cơ sở dữ liệu trở nên cực kỳ dễ dàng. Khi sử dụng Eloquent, mỗi bảng cơ sở dữ liệu có một “Model” tương ứng dùng để tương tác với bảng đó. Các Eloquent models cho phép thêm, truy xuất, cập nhật và xóa các bản ghi khỏi các bảng cơ sở dữ liệu. Có thể tìm thêm thông tin về Laravel Eloquent tại đây: <https://laravel.com/docs/11.x/eloquent>.

Các Laravel models được đặt trong thư mục app/Http/Models. Trong thư mục đó, sẽ tìm thấy tập tin User.php chứa model lớp User được xác định trước của Laravel. Bây giờ chúng ta sẽ tập trung vào model Product, vì vậy hãy tạo nó.

5.3.2. TẠO PRODUCT MODEL

Trong Terminal, đi tới thư mục dự án và thực hiện như sau:

Terminal

```
php artisan make:model Product
```

Sẽ thấy tập tin Product.php bên trong thư mục app/Models. Hãy cùng phân tích code model Product.

Code

```
<?php
namespace App\Models;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
class Product extends Model
{
    use HasFactory;
}
```

Chúng ta có một lớp Product mở rộng lớp Laravel Eloquent Model của Laravel. Lớp Laravel Eloquent Model sẽ cung cấp cho lớp Product một tập hợp các phương thức và thuộc tính hữu ích. Những phương thức đó sẽ giúp việc giao tiếp với cơ sở dữ liệu trở nên cực kỳ dễ dàng. Do đó, nó sử dụng đặc điểm HasFactory.

Chúng ta sẽ thảo luận một số vấn đề với những models “empty” này sau.

5.3.3. NHỮNG LƯU Ý QUAN TRỌNG VỀ ELOQUENT

Hãy kiểm tra một số lưu ý khi sử dụng Laravel Eloquent.

- Eloquent cũng sẽ giả định rằng bảng cơ sở dữ liệu tương ứng của mỗi model có một cột khóa chính có tên id. Đối với tất cả các migrations, chúng ta sẽ sử dụng phương thức id xác định cột id.
- Eloquent sẽ giả sử model Product lưu trữ các bản ghi trong bảng products (lưu ý ký tự “s”). Quy ước này áp dụng cho tất cả các models.
- Theo mặc định, Eloquent mong đợi các cột create_at và update_at tồn tại trên bảng cơ sở dữ liệu tương ứng của model. Eloquent sẽ tự động đặt các giá trị của cột này khi model

được tạo hoặc cập nhật. Đối với tất cả các migrations, chúng ta sẽ sử dụng phương thức timestamps để tạo các cột này.

5.3.4. CÁC PHƯƠNG THỨC CHÍNH

Hãy cùng thảo luận về một số phương thức chính mà Eloquent cung cấp cho các model của chúng ta.

1. `Product::all()` : truy xuất tất cả các bản ghi sản phẩm.
2. `Product::find(1)`: lấy sản phẩm có id 1.
3. `Product::findOrFail(1)`: nó tương tự như cái trước, nhưng nó sẽ đưa ra một ngoại lệ nếu không tìm thấy kết quả nào.
4. `Product::create(['name' => 'TV', ...])`: tạo một bản ghi mới trong cơ sở dữ liệu. Phải truyền một mảng kết hợp với dữ liệu được chỉ định, không bắt buộc phải có id vì nó được tạo tự động.
5. `Product::destroy(1)`: xóa sản phẩm có id 1.

Chúng ta sẽ sử dụng một số phương thức trên đó trong các phần tiếp theo. Hiện tại, hãy sửa đổi ứng dụng để trích xuất dữ liệu sản phẩm từ cơ sở dữ liệu MySQL.

5.3.4. LIỆT KÊ CÁC SẢN PHẨM VỚI DỮ LIỆU CƠ SỞ DỮ LIỆU

Để trích xuất dữ liệu từ cơ sở dữ liệu MySQL, chúng ta chỉ cần sửa đổi `ProductController`.

Sửa đổi `ProductController`

Trong `app/Http/Controllers/ProductController.php`, hãy thay đổi phần in đậm, màu đỏ sau.

Code

```
<?php
namespace App\Http\Controllers;
use App\Models\Product;
use Illuminate\Http\Request;

class ProductController extends Controller
{
    public static $products = [
        ["id"=>"1", "name"=>"TV", "description"=>"Best TV", "image"=>"TV.jpg",
        "price"=>"2000"],
        ["id"=>"2", "name"=>"iPhone", "description"=>"Best iPhone", "image"=>
        "iPhone.jpeg", "price"=>"1500"],
        ["id"=>"3", "name"=>"Chromecast", "description"=>"Best Chromecast",
        "image"=>"Chromecast.jpeg", "price"=>"300"],
        ["id"=>"4", "name"=>"Glasses", "description"=>"Best Glasses", "image"=>
        "Glasses.jpeg", "price"=>"500"]
    ];
}
```

```
public function index()
{
    $viewData = [];
    $viewData["title"] = "Products - Online Store";
    $viewData["subtitle"] = "Danh sách sản phẩm.";
    $viewData["products"] = ProductController::$products;
    $viewData["products"] = Product::all();
    return view('product.index')->with("viewData", $viewData);
}

public function show($id)
{
    $viewData = [];
    $product = Product::findOrFail($id);
    $product = ProductController::$products[$id-1];
    $viewData["title"] = $product["name"]." - Online Store";
    $viewData["subtitle"] = $product["name"]." - Thông tin sản phẩm.";
    $viewData["product"] = $product;
    return view('product.show')->with("viewData", $viewData);
}
}
```

Hãy phân tích code trước đó theo từng phần.

Code

```
use App\Models\Product;
```

Chúng ta sử dụng Product model, kết nối với bảng cơ sở dữ liệu.

Code

```
public static $products = [
    ["id"=>"1", "name"=>"TV", "description"=>"Best TV", "image"=>"TV.jpg",
    "price"=>"2000"],
    ["id"=>"2", "name"=>"iPhone", "description"=>"Best iPhone", "image"=>
    "iPhone.jpeg", "price"=>"1500"],
    ["id"=>"3", "name"=>"Chromecast", "description"=>"Best Chromecast",
    "image"=>"Chromecast.jpeg", "price"=>"300"],
    ["id"=>"4", "name"=>"Glasses", "description"=>"Best Glasses", "image"=>
    "Glasses.jpeg", "price"=>"500"]
];
```

Chúng ta xóa giá trị thuộc tính giả của sản phẩm vì chúng ta không cần nó nữa. Thay vào đó, chúng ta sẽ truy xuất dữ liệu sản phẩm từ cơ sở dữ liệu.

Code

```
$viewData["products"] = Product::all();
```

Chúng ta có `Product::all()` trong phương thức `index`, phương thức này lấy các sản phẩm từ cơ sở dữ liệu.

Code

```
$product = Product::findOrFail($id);
```

Trong phương thức `show`, chúng ta có `Product::findOrFail($id)`, truy xuất một sản phẩm cụ thể dựa trên id của nó. `findOrFail` có thể đưa ra `ModelNotFoundException` (tức là khi truyền một id không hợp lệ). Nếu `ModelNotFoundException` không bị bắt, phản hồi HTTP 404 sẽ tự động được gửi lại cho máy khách.

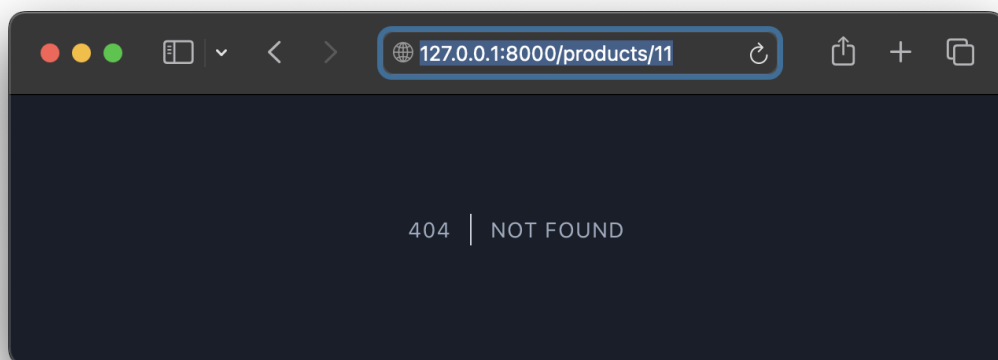
Chạy ứng dụng

Trong Terminal, đi tới thư mục dự án và thực hiện như sau:

Terminal

```
php artisan serve
```

Đi tới tuyến ("/products") và sẽ thấy các sản phẩm được truy xuất từ cơ sở dữ liệu MySQL. Hãy thử truy cập một sản phẩm cụ thể có id không tồn tại (ví dụ: "/products/11"). Ứng dụng sẽ hiển thị lỗi 404 (xem Hình 5-9).



Hình 5-9. Truy cập một sản phẩm không tồn tại.

5.4. TÁI CẤU TRÚC DANH SÁCH SẢN PHẨM

Nhiều thứ có thể được cải thiện trong code trước đó. Ví dụ: chúng ta sẽ cấu trúc lại `Product`

model, ProductController và product views. Những thay đổi này sẽ làm cho code dễ bảo trì, dễ hiểu và rõ ràng hơn. Một lần nữa, nhiều thay đổi trong số này không chỉ áp dụng cho dự án Laravel mà còn là những mẹo chung có thể được áp dụng trong hầu hết các framework MVC.

5.4.1. MODEL ATTRIBUTES

Mở tập tin Product model và thử đoán chỉ bằng cách xem tập tin đó có Product model attributes hay không. Chúng ta có một vấn đề ở đây. Chúng ta không thể suy ra các thuộc tính của Product model. Chúng ta phải xem migrations “tạo bảng sản phẩm” hoặc mở phpMyAdmin và đi tới bảng sản phẩm chỉ để tìm câu trả lời đó. Theo quan điểm cá nhân, cách thiết kế Product model sẽ ảnh hưởng đến mức độ dễ hiểu của dự án. Nhiều framework MVC khác, chẳng hạn như Django (Python) hoặc Spring (Java), xác định rõ ràng các product attributes của models, nhưng đó không phải là trường hợp của Laravel.

Hãy cấu trúc lại Product model. Trong app/Models/Product.php, hãy thay đổi phần in đậm, màu đỏ sau.

Code

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Product extends Model
{
    use HasFactory;
    /**
     * PRODUCT ATTRIBUTES
     * $this->attributes['id'] - int - contains the product primary key (id)
     * $this->attributes['name'] - string - contains the product name
     * $this->attributes['description'] - string - contains the product
description
     * $this->attributes['image'] - string - contains the product image
     * $this->attributes['price'] - int - contains the product price
     * $this->attributes['created_at'] - timestamp - contains the product
creation date
     * $this->attributes['updated_at'] - timestamp - contains the product
update date
     */
}
```

Hãy phân tích code trên.

• Chúng ta đã loại bỏ việc sử dụng đặc điểm HasFactory. Model factories rất hữu ích trong việc tự động hóa việc tạo bản ghi model giả trong cơ sở dữ liệu. Ví dụ: có nhớ khi chúng ta thực hiện một số truy vấn SQL để thêm một số sản phẩm vào cơ sở dữ liệu không? Các nhà máy kiểu mẫu có thể tự động hóa quy trình này cho chúng ta. Có thể tìm thêm thông tin về các model factories tại đây: <https://laravel.com/docs/11.x/database-testing#defining-model-factories>.

• Cuối cùng, chúng ta đã đưa vào khối PHP comment để chỉ định các thuộc tính có sẵn cho Product model. Lập trình viên mở Product model có thể dễ dàng tìm thấy các thuộc tính có sẵn.

Chúng ta đã khai báo các thuộc tính của model ở dạng `$this->attributes['id']`. Đó là bởi vì Laravel Eloquent lưu trữ các thuộc tính của model trong thuộc tính mảng lớp có tên là `$attributes`.

Kiểm tra đoạn code sau để hiểu cách hoạt động của các thuộc tính của model Laravel Eloquent.

Code

```
$product = Product::findOrFail(1);
echo $product->name; # prints the product's name
echo $product["name"]; # prints the product's name
```

Laravel Eloquent cung cấp hai cách truy cập các thuộc tính của model. Dạng thuộc tính đối tượng (`$product->name`) và dạng mảng kết hợp (`$product["name"]`). Cả hai tùy chọn đều truy cập nội bộ vào model Sản phẩm và tìm kiếm dữ liệu `$this->attributes['name']`. Nếu thuộc tính đó không tồn tại, nó sẽ trả về null. Nếu không, nó trả về giá trị được lưu trữ trong thuộc tính mảng lớp (`$attributes`). Chúng ta sẽ không sử dụng bất kỳ hình thức truy cập thuộc tính nào của model này. Chúng ta sẽ sử dụng một cách tốt hơn, nhưng trước tiên, hãy xem tại sao chúng ta cần một cách tốt hơn để truy cập các thuộc tính của model.

5.4.2. KHÔNG CÓ GETTERS VÀ SETTERS

Hãy phân tích code hiện tại của chúng ta. Nó là một đoạn trích từ tập tin `ProductController.php` của chúng ta.

Code

```
public function show($id)
{
    $viewData = [];
    $product = Product::findOrFail($id);
    $viewData["title"] = $product["name"]." - Online Store";
    $viewData["subtitle"] = $product["name"]." - Thông tin sản phẩm.";
    $viewData["product"] = $product;
    return view('product.show')->with("viewData", $viewData);
}
```

Chúng ta đang truy cập tên của sản phẩm thông qua biểu mẫu mảng kết hợp (\$product["name"]). Bây giờ, hãy phân tích view sản phẩm/show.blade.php.

Code

```
<div class="card-body">
    <h5 class="card-title">
        {{$viewData["product"]["name"]}}({{$viewData["product"]["price"]}})
    </h5>
    <p class="card-text">{{ $viewData["product"]["description"] }}</p>
    <p class="card-text"><small class="text-muted">ThemvaoGiohang.</small></p>
</div>
```

Một lần nữa, chúng ta đang truy cập tên của sản phẩm thông qua dạng mảng kết hợp (\$viewData["product"]["name"]).

Vấn đề khi truy cập dữ liệu thực thể theo cách này là gì? Hãy tưởng tượng rằng: “Chúng ta cần hiển thị tên tất cả các sản phẩm bằng chữ in hoa trong toàn bộ ứng dụng”. Đó là một vấn đề lớn khi chúng ta trích xuất tên sản phẩm qua nhiều view và controllers khác nhau. Yêu cầu đơn giản này sẽ yêu cầu chúng ta sửa đổi một số view và controllers. Bây giờ, hãy xem chúng ta phải làm gì để đạt được yêu cầu đó.

Chúng ta nên sửa đổi ProductController.php theo cách này (không thực hiện thay đổi này, nó chỉ được sử dụng để minh họa cho tình huống này):

Code

```
public function show($id)
{
    $viewData = [];
    $product = Product::findOrFail($id);
    $viewData["title"] = strtoupper($product["name"]) . " - Online Store";
    $viewData["subtitle"] = strtoupper($product["name"]) . " - Thông tin SP.";
    $viewData["product"] = $product;
    return view('product.show')->with("viewData", $viewData);
}
```

Và view product/show.blade.php theo cách này (không thực hiện thay đổi này, nó chỉ được sử dụng để minh họa cho kịch bản này):

Code

```
<div class="card-body">
    <h5 class="card-title">
        {{ strtoupper($viewData["product"]["name"]) }} ({{$viewData["product"]["price"] }})
    </h5>
```

```

<p class="card-text">{{ $viewData["product"]["description"] }}</p>
<p class="card-text"><small class="text-muted"> Thêm vào Gio
hang.</small></p>
</div>

```

Có hai vấn đề quan trọng với chiến lược này. (i) Chúng ta có nhiều code trùng lặp trong toàn bộ ứng dụng (hàm strtoupper được sử dụng ở một số nơi). Và (ii) chúng ta phải kiểm tra tất cả các controllers, tất cả các views và có thể hàng chục hoặc hàng trăm tập tin để kiểm tra xem chúng ta cần áp dụng hàm strtoupper ở đâu. Nó không thể bảo trì được. Vì vậy, hãy cấu trúc lại code của chúng ta để triển khai chiến lược tốt hơn.

5.4.3. DỰ ÁN VỚI GETTERS VÀ SETTERS

Tái cấu trúc Model Product

Đầu tiên, hãy cấu trúc lại model Product của chúng ta. Trong app/Models/Product.php, hãy thay đổi phần màu đỏ, in đậm sau.

Code

```

<?php

namespace App\Models;
use Illuminate\Database\Eloquent\Model;
class Product extends Model
{
    /**
     * PRODUCT ATTRIBUTES
     * $this->attributes['id'] - int - contains the product primary key (id)
     * $this->attributes['name'] - string - contains the product name
     * $this->attributes['description'] - string - contains the product
description
     * $this->attributes['image'] - string - contains the product image
     * $this->attributes['price'] - int - contains the product price
     * $this->attributes['created_at'] - timestamp - contains the product
creation date
     * $this->attributes['updated_at'] - timestamp - contains the product
update date
    */
    public function getId()
    {
        return $this->attributes['id'];
    }
    public function setId($id)
    {
        $this->attributes['id'] = $id;
    }
}

```

```
public function getName()
{
    return $this->attributes['name'];
}
public function setName($name)
{
    $this->attributes['name'] = $name;
}
public function getDescription()
{
    return $this->attributes['description'];
}
public function setDescription($description)
{
    $this->attributes['description'] = $description;
}
public function getImage()
{
    return $this->attributes['image'];
}
public function setImage($image)
{
    $this->attributes['image'] = $image;
}
public function getPrice()
{
    return $this->attributes['price'];
}
public function setPrice($price)
{
    $this->attributes['price'] = $price;
}
public function getCreatedAt()
{
    return $this->attributes['created_at'];
}
public function setCreatedAt($createdAt)
{
    $this->attributes['created_at'] = $createdAt;
}
public function getUpdatedAt()
{
    return $this->attributes['updated_at'];
}
public function setUpdatedAt($updatedAt)
{

```

```

        $this->attributes['updated_at'] = $updatedAt;
    }
}

```

Đối với mỗi Product attribute, chúng ta xác định getter và setter tương ứng. Chúng ta sẽ sử dụng getters và setters để truy cập và sửa đổi các thuộc tính model của mình. Nó có nhiều ưu điểm mà chúng ta sẽ nói đến sau.

Tái cấu trúc ProductController

Trong app/Http/Controllers/ProductController.php, hãy thay đổi phần in đậm, màu đỏ sau.

Code

```

public function show($id)
{
    $viewData = [];
    $product = Product::findOrFail($id);
    $viewData["title"] = $product->getName(). " - Online Store";
    $viewData["subtitle"] = $product->getName(). " - Thông tin sản phẩm.";
    $viewData["product"] = $product;
    return view('product.show')->with("viewData", $viewData);
}

```

Bây giờ, chúng ta truy cập các thuộc tính sản phẩm thông qua getters và setters tương ứng.

Tái cấu trúc view product/index

Trong resources/views/product/index.blade.php, hãy thay đổi phần in đậm, màu đỏ sau.

Code

```

<div class="row">
    @foreach ($viewData["products"] as $product)
        <div class="col-md-4 col-lg-3 mb-2">
            <div class="card">
                
                <div class="card-body text-center">
                    <a href="{{ route('product.show', ['id' => $product->getId()])
}}" class="btn bg-primary text-white">{{ $product->getName() }}</a>
                </div>
            </div>
        </div>
    @endforeach
</div>

```

Giống như controllers trước đó, chúng ta truy cập các thuộc tính của sản phẩm thông qua các getters tương ứng.

Tái cấu trúc view product/show

Trong Resources/views/product/show.blade.php, hãy thay đổi phần in đậm, màu đỏ sau.

Code

```
<div class="card mb-3">
    <div class="row g-0">
        <div class="col-md-4">
            getImage()) }}"
            class="img-fluid rounded-start">
        </div>
        <div class="col-md-8">
            <div class="card-body">
                <h5 class="card-title">
                    {{ $viewData["product"]->getName() }} ({{
                    $viewData["product"]->getPrice() }})
                </h5>
                <p class="card-text">{{ $viewData["product"]->getDescription()
            }}</p>
                <p class="card-text"><small class="text-muted">Thêm vào Gio
            hang.</small></p>
            </div>
        </div>
    </div>
</div>
```

Chúng ta truy cập các thuộc tính sản phẩm thông qua getters.

Phân tích getters và setters

Hiện tại, ứng dụng trông giống nhau. Chúng ta chỉ sửa đổi cách chúng ta truy cập các thuộc tính của model. Vì vậy, lợi thế là gì? Hãy xem lại yêu cầu: “chúng ta cần hiển thị tên tất cả các sản phẩm bằng chữ in hoa trên toàn bộ ứng dụng”.

Trong trường hợp này, chúng ta chỉ cần sửa đổi tập tin model Product. Cụ thể là phương thức getName. Hãy xem sửa đổi (có thể áp dụng thay đổi sau hoặc để nguyên như vậy).

Code

```
public function getName()
{
    return strtoupper($this->attributes['name']);
}
```

Nếu chạy ứng dụng, sẽ thấy tên tất cả sản phẩm đều được viết hoa. Chúng ta chỉ yêu cầu một thay đổi duy nhất ở một vị trí cụ thể. Đó là sức mạnh của việc sử dụng getters hoặc setters. Việc định nghĩa và sử dụng getters và setters đảm bảo một điểm truy cập duy nhất tới các thuộc tính của model. Đó là một phần của cái mà một số người gọi là đóng gói, một trong ba trụ cột của lập trình hướng đối tượng.

Luôn cố gắng truy cập các thuộc tính model của thông qua getters và setters. Nó sẽ giúp việc thêm các chức năng trong tương lai trở nên dễ dàng hơn. Thậm chí có thể bao gồm một quy tắc mới nói rằng các thuộc tính của model phải được truy cập thông qua getters và setters tương ứng của chúng (trong tài liệu quy tắc kiến trúc).

Laravel cung cấp một cách khác để triển khai getters và setters. Laravel gọi chúng là Accessors và Mutators (có thể đọc thêm về nó tại đây: <https://laravel.com/docs/11.x/eloquent-mutators#accessors-and-mutators>). Chúng ta không thích chiến lược này vì một lý do duy nhất. Khi sử dụng Accessors và Mutators cũng như quyền truy cập vào các thuộc tính model, tiếp tục sử dụng biểu mẫu ban đầu (tức là `$product->name`) so với biểu mẫu getter cổ điển (`$product->getName()`). Chúng ta thích cách thứ hai hơn vì nó nói rõ ràng rằng truy cập dữ liệu thông qua một phương thức lớp. Nó cải thiện tính dễ hiểu của code. Nếu không, người lập trình phải vào tập tin model để kiểm tra xem Accessor có được triển khai hay không.

Chúng ta sẽ sử dụng getters và setters cổ điển cho phần còn lại của ứng dụng.

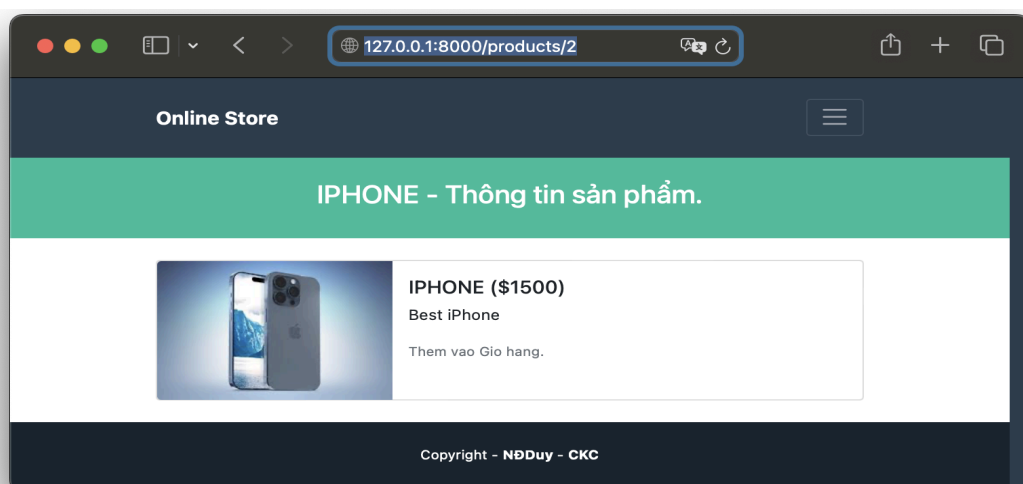
Chạy ứng dụng

Trong Terminal, đi tới thư mục dự án và thực hiện như sau:

Terminal

```
php artisan serve
```

Ứng dụng phải hoạt động bình thường. Nếu áp dụng sửa đổi trong phương thức `getName`, sẽ thấy tên tất cả các sản phẩm được viết hoa (xem Hình 5-10).



Hình 5-10. Truy cập một sản phẩm bằng phương thức `getName` đã sửa đổi.